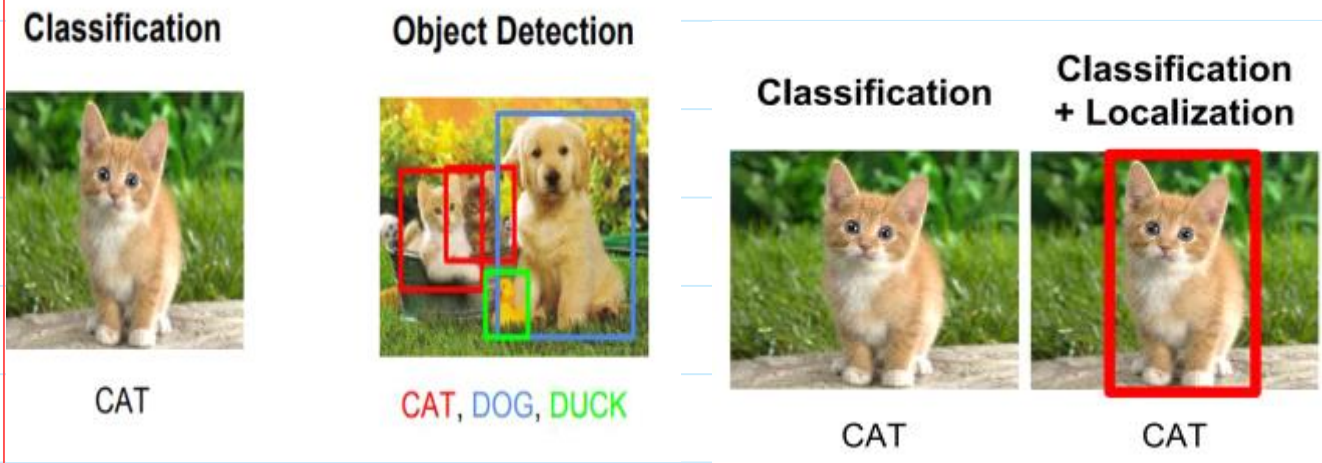
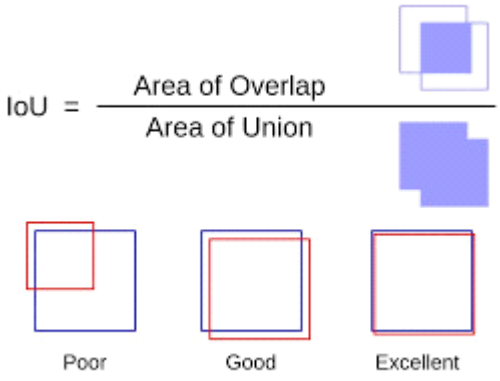
딥러닝 영상처리- Lecture Week11-2

Object Detection

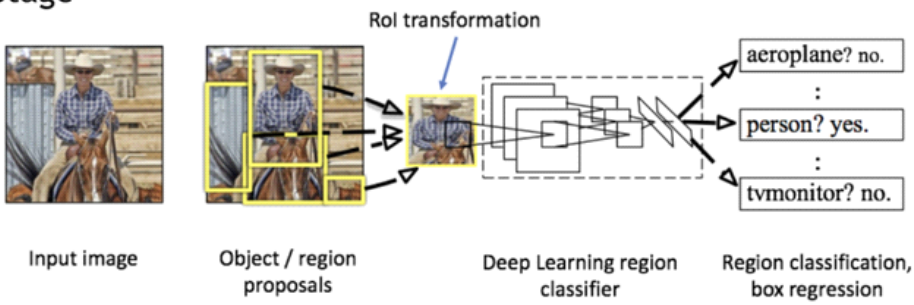


- 컴퓨터 비전 분야에서, 이미지나 비디오에서 객체를 탐지하고, 해당 객체의 위치와 크기를 식별하는 기술
- 다수의 객체를 감지하고, 분류하는 작업을 포함
- 객체의 위치를 찾는다 --> 해당 객체의 경계 상자(Bounding box)를 찾는다
- 자율 주행, 보안 카메라 시스템, 의료 영상 등 다양한 분야에서 활용

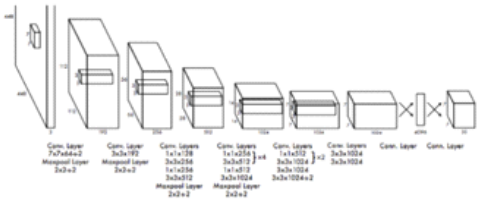
<IoU , Intersection over Union>

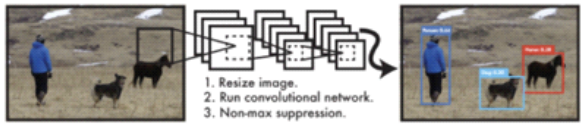


Many stage



One stage





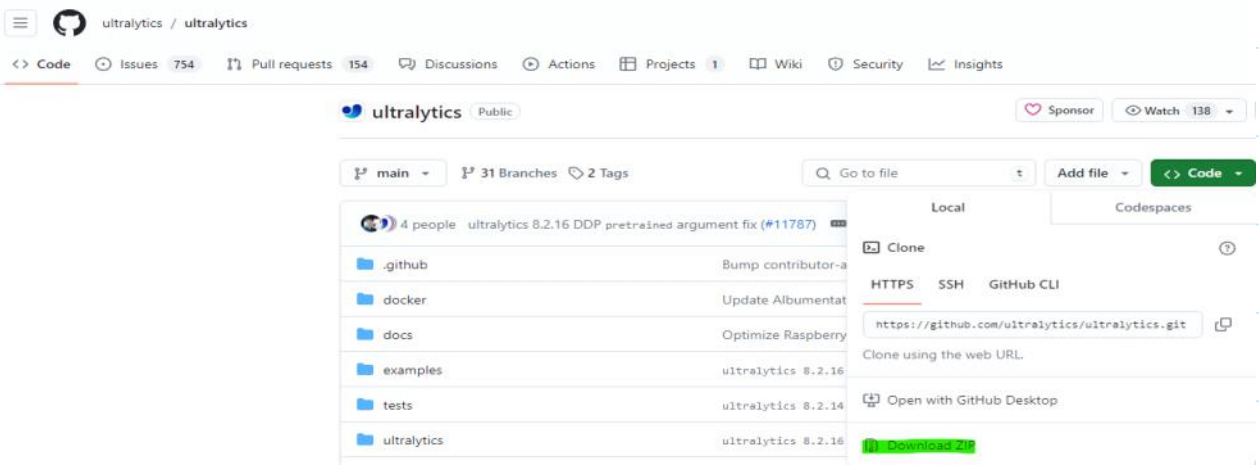
Redmond et al. You Only Look Once:
Unified Real-time Object Detection. In CVPR 2016

YOLO-V11

Github 링크 : https://github.com/ultralytics/ultralytics

환경 준비

1. 코드 다운로드 후 압축 풀기



See below for quickstart installation and usage examples. For comprehensive guidance on training, validation, prediction, and deployment, refer to our full Ultralytics Docs.

▼ Install

Install the `ultralytics` package, including all requirements, in a Python>=3.8 environment with PyTorch>=1.8.

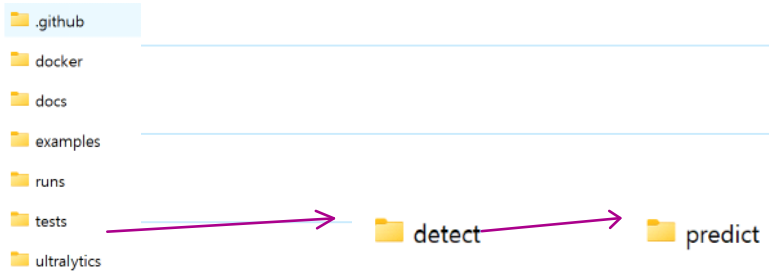
2. conda create -n yolov11 python==3.9 -y

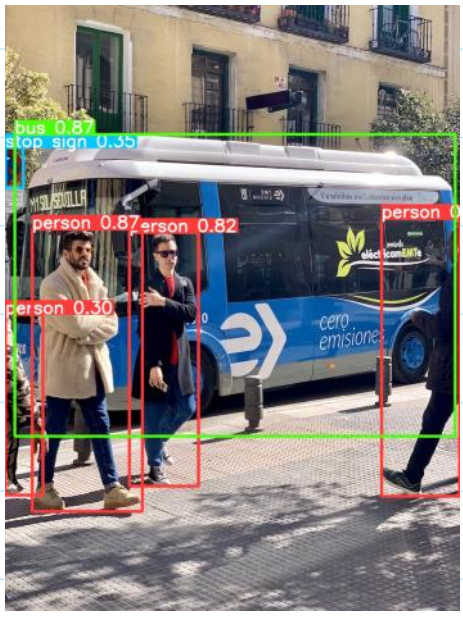
3. activate yolov11

4. pip install ultralytics

동작 확인

yolo predict model=yolo11n.pt source='https://ultralytics.com/images/bus.jpg'






```
image 1/1 D:₩lecture_note₩2025-1-DIP₩bus.jpg: 640x480 4 persons, 1 bus, 150.0ms
Speed: 3.7ms preprocess, 150.0ms inference, 2.7ms postprocess per image at shape (1, 3, 640, 480)
Results saved to runs₩detect₩predict
 Learn more at https://docs.ultralytics.com/modes/predict
```

ultralytics-main\ultralytics\cfg\default.yaml

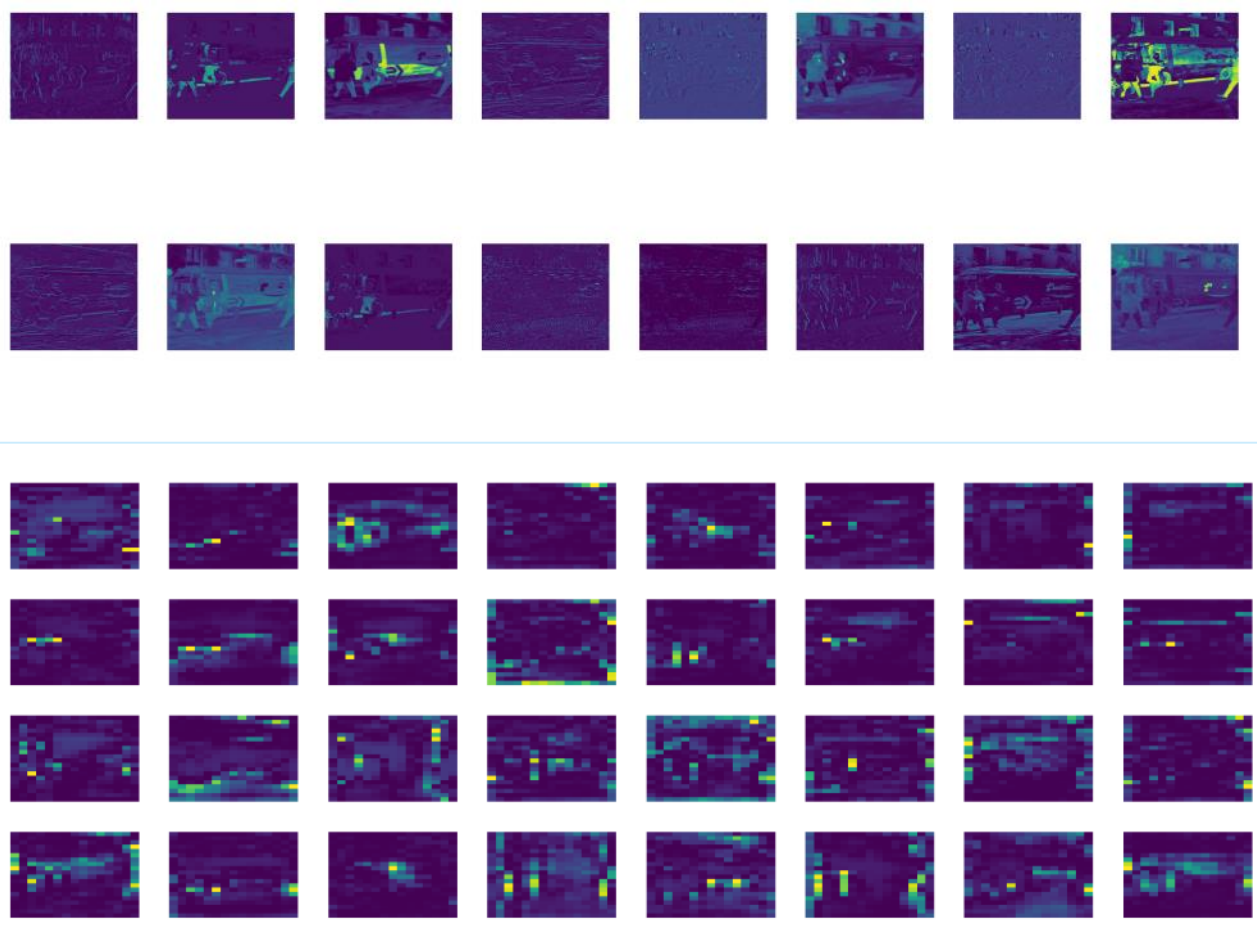
```
# Train settings -------------------------------------------------
model: # (str, optional) path to model file, i.e. yolov8n.pt, y
data: # (str, optional) path to data file, i.e. coco8.yaml
epochs: 100 # (int) number of epochs to train for
time: # (float, optional) number of hours to train for, overrid
patience: 100 # (int) epochs to wait for no observable improvem
batch: 16 # (int) number of images per batch (-1 for AutoBatch)
imgsz: 640 # (int | list) input images size as int for train an
```

```
# Predict settings ---------------------------------------------------------------------------------------
source: # (str, optional) source directory for images or videos
vid_stride: 1 # (int) video frame-rate stride
stream_buffer: False # (bool) buffer all streaming frames (True) or return the most recent frame (False)
visualize: False # (bool) visualize model features
augment: False # (bool) apply image augmentation to prediction sources
agnostic_nms: False # (bool) class-agnostic NMS
classes: # (int | list[int], optional) filter results by class, i.e. classes=0, or classes=[0,2,3]
retina_masks: False # (bool) use high-resolution segmentation masks
embed: # (list[int], optional) return feature vectors/embeddings from given layers

# Visualize settings -------------------------------------------------------------------------------------
show: False # (bool) show predicted images and videos if environment allows
save_frames: False # (bool) save predicted individual video frames
save_txt: False # (bool) save results as .txt file
save_conf: False # (bool) save results with confidence scores
save_crop: False # (bool) save cropped images with results
show_labels: True # (bool) show prediction labels, i.e. 'person'
show_conf: True # (bool) show prediction confidence, i.e. '0.99'
show_boxes: True # (bool) show prediction boxes
line_width: # (int, optional) line width of the bounding boxes. Scaled to image size if None.
```

yolo predict model=yolo11n.pt source='https://ultralytics.com/images/bus.jpg'

yolo predict model=yolo11n.pt source='https://ultralytics.com/images/bus.jpg' show=True
yolo predict model=yolo11n.pt source='https://ultralytics.com/images/bus.jpg' visualize=True



```
yolo predict model=yolo11n.pt source='https://ultralytics.com/images/bus.jpg'  save_txt=True
```

bus.txt

```
5 0.493844 0.443415 0.978224 0.462082
0 0.914091 0.589604 0.171349 0.448036
0 0.176979 0.603593 0.236909 0.46725
0 0.350325 0.587558 0.149887 0.418284
0 0.0425353 0.661309 0.085017 0.292862
```

<class_id> <center_x> <center_y> <width> <height>

`absolute_center_x = center_x * image_width`

`absolute_center_y = center_y * image_height`

`absolute_width = width * image_width`

`absolute_height = height * image_height`

```python
import cv2

image = cv2.imread('bus.jpg')

boxes = [
    (5, 409.40, 499.50, 785.25, 538.81),
    (0, 146.62, 651.29, 195.98, 503.24),
    (0, 739.06, 628.99, 139.59, 495.41),
    (0, 283.05, 631.59, 123.18, 451.60),
    (11, 16.18, 289.85, 32.22, 70.39),
    (0, 33.55, 712.47, 67.14, 322.93)
]

for box in boxes:
    class_id, center_x, center_y, width, height = box
    top_left_x = int(center_x - width / 2)
    top_left_y = int(center_y - height / 2)
    bottom_right_x = int(center_x + width / 2)
    bottom_right_y = int(center_y + height / 2)

    # 네모 박스 그리기 (파란색, 두께 2)
    cv2.rectangle(image, (top_left_x, top_left_y), (bottom_right_x, bottom_right_y), (255, 0, 0), 2)
    # 클래스 ID 텍스트 추가 (빨간새, 두께 2)
    cv2.putText(image, str(class_id), (top_left_x, top_left_y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 2)

# 이미지 저장 또는 표시
cv2.imwrite('bus_draw_boxes.jpg', image)
```

yolo detect train data=coco8.yaml model=yolo11n.yaml epochs=100 imgsz=640

https://docs.ultralytics.com/modes/train/